SOCIALIST REPUBLIC OF VIETNAM

Independence - Freedom - Happiness

NATIONAL ECONOMICS UNIVERSITY (NEU)

FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE (FDA)

UNIVERSITY LOGO (NEU) FACULTY LOGO (FDA)

MYSQL DATABASE PRACTICAL PROJECT REPORT

TOPIC 01: TUTORING CENTER MANAGEMENT SYSTEM

`Database Schema: tutoring_mini`

PROJECT TEAM MEMBERS

No.	Student Full Name	Student ID	Assigned Responsibilities
1	**Pham Ngoc Danh**	**11251809**	Database schema design (`schema.sql`), analytical VIEWs (`views.sql`) & Report compilation
2	**Nguyen Manh Dung**	**11252140**	Sample dataset design (`seed.sql`), integrity verification & Queries 1 – 5
3	**Hoang Duc Duong**	**11252228**	Development & testing of Queries 6 – 10 in `queries.sql` & GitHub repository packaging

HANOI — 2026

SECTION 1. OVERVIEW & BUSINESS PROBLEM ANALYSIS

1.1. Objectives and Business Context

The **Tutoring Center Management System (`tutoring_mini`)** relational database is engineered to standardize and automate the end-to-end operational workflow of a modern academic tutoring center: from managing the catalog of academic subjects (`subjects`), tutor qualifications and subject assignments (`tutors`), and student profiles (`students`), to coordinating lesson schedules and lifecycle statuses (`lessons`), recording post-lesson academic evaluations and confirmed tuition fees (`lesson_reports`), and tracking single or multi-installment tuition payments (`payments`).

1.2. Core Business Rules & Data Integrity Constraints

5 Mandatory Business & Integrity Rules in tutoring_mini

1. **Tutor Subject Assignment (1 – N):** Each academic subject has a unique name (`UNIQUE`). Each tutor specializes in teaching one specific subject and holds a unique institutional `email` address.
2. **Lesson Status Lifecycle (`ENUM`):** Every scheduled lesson between a student and a tutor must be in one of three states: `'Pending'` (Scheduled/Awaiting completion), `'Completed'` (Finished), or `'Cancelled'` (Cancelled).
3. **Post-Lesson Reporting Rule (Strict 1 – 1 Relationship):** Only lessons with status `'Completed'` may have a `lesson_reports` entry, and **every** `'Completed'` lesson must have exactly one report (enforced via `lesson_id UNIQUE` , positive duration `duration_minutes > 0` , and non-negative tuition fee `fee >= 0`).
4. **Tuition Collection & Installment Payments (1 – N Relationship):** Payments are collected only for lessons that already have a confirmed report (`lesson_reports`). A lesson may be paid in full at once, paid across multiple installments (multiple rows in `payments` sharing the same `lesson_id`), or remain partially/fully unpaid, provided the cumulative amount paid (`SUM(amount)`) for any `lesson_id` never exceeds its confirmed tuition fee (`fee`).
5. **Anti-Fanout / Chasm Trap Prevention:** Because `lesson_reports` has a 1–1 relationship with `lessons` whereas `payments` has a 1–N relationship with `lessons` , any query or view that simultaneously aggregates confirmed fees (`fee`) and paid amounts (`amount`) must pre-aggregate `payments` by `lesson_id` inside a Subquery/CTE before joining to `lesson_reports` so that tuition fees are never multiplied.

SECTION 2. DATABASE SCHEMA DESIGN (PART A — SCHEMA.SQL)

2.1. Entity-Relationship Diagram (ERD)

The `tutoring_mini` database is normalized to Third Normal Form (**3NF**), eliminating data redundancy and update, insertion, or deletion anomalies. The relationships among the 6 tables are illustrated in Figure 2.1:

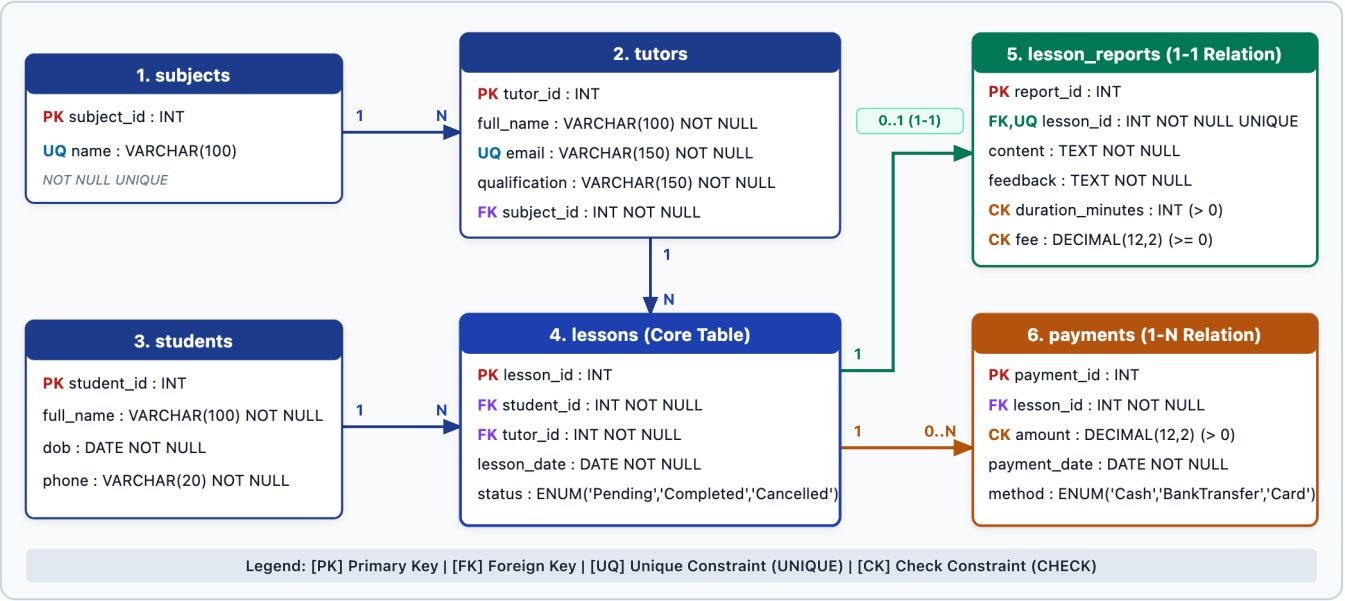


Figure 2.1: Entity-Relationship Diagram (ERD) of the Tutoring Center Management Database (`tutoring_mini`).

2.2. Detailed Specification of the 6 Tables and Integrity Constraints

Table Name	Columns (Primary Key First)	Primary / Foreign Keys	Integrity Constraints (UNIQUE, CHECK, ENUM) & Design Rationale
`subjects`	`subject_id`, `name`	**PK:** `subject_id`	`name VARCHAR(100) NOT NULL UNIQUE` — Prevents duplicate subject names in the academic catalog.
`tutors`	`tutor_id`, `full_name`, `email`, `qualification`, `subject_id`	**PK:** `tutor_id` **FK:** `subject_id` → `subjects(subject_id)`	All columns are `NOT NULL`; `email UNIQUE` prevents duplicate tutor accounts; `FK` ensures every tutor is assigned to a valid subject.
`students`	`student_id`, `full_name`, `dob`, `phone`	**PK:** `student_id`	`full_name`, `dob (DATE)`, and `phone` are all `NOT NULL` — Stores complete identity and contact records for each student.
`lessons`	`lesson_id`, `student_id`, `tutor_id`, `lesson_date`, `status`	**PK:** `lesson_id` **FK1:** `student_id` → `students` **FK2:** `tutor_id` → `tutors`	`lesson_date DATE NOT NULL`; `status ENUM('Pending', 'Completed', 'Cancelled') NOT NULL` — Strictly restricts lesson status values at the storage engine level.
`lesson_reports`	`report_id`, `lesson_id`, `content`, `feedback`, `duration_minutes`, `fee`	**PK:** `report_id` **FK + UQ:** `lesson_id` → `lessons(lesson_id)`	`lesson_id UNIQUE` enforces a strict **1–1** relationship (at most one report per lesson); `CHECK (duration_minutes > 0)` and `CHECK (fee >= 0)` enforce valid numeric domains.
`payments`	`payment_id`, `lesson_id`, `amount`, `payment_date`, `method`	**PK:** `payment_id` **FK:** `lesson_id` → `lessons(lesson_id)`	No `UNIQUE` constraint on `lesson_id` to allow multi-installment payments (1–N); `CHECK (amount > 0)`; `method ENUM('Cash', 'BankTransfer', 'Card') NOT NULL`.

Table 2.1: Summary of the 6-table schema architecture and integrity constraints in `schema.sql`.

MySQL Technical Note: In MySQL 8.0/9.0, `CHECK` constraints cannot contain subqueries referencing other tables. Therefore, single-table domain rules (`duration_minutes > 0`, `fee >= 0`, `amount > 0`) are enforced directly via declarative `CHECK` constraints in `schema.sql`, while cross-table business rules (only `Completed` lessons have reports, and cumulative `amount <= fee`) are verified via automated reconciliation queries at the end of `seed.sql`.

2.3. Complete Source Code of `schema.sql`

```
DROP DATABASE IF EXISTS tutoring_mini;
CREATE DATABASE tutoring_mini
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE tutoring_mini;

-- 1. Table subjects: Catalog of academic subjects
CREATE TABLE subjects (
    subject_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

-- 2. Table tutors: Tutor profiles and subject assignments
CREATE TABLE tutors (
    tutor_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    qualification VARCHAR(150) NOT NULL,
    subject_id INT NOT NULL,
    CONSTRAINT fk_tutors_subject
        FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
) ENGINE=InnoDB;

-- 3. Table students: Student profiles and contact details
CREATE TABLE students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    dob DATE NOT NULL,
    phone VARCHAR(20) NOT NULL
) ENGINE=InnoDB;

-- 4. Table lessons: Lesson schedule and lifecycle status
CREATE TABLE lessons (
    lesson_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    tutor_id INT NOT NULL,
    lesson_date DATE NOT NULL,
    status ENUM('Pending', 'Completed', 'Cancelled') NOT NULL,
    CONSTRAINT fk_lessons_student
        FOREIGN KEY (student_id) REFERENCES students(student_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_lessons_tutor
        FOREIGN KEY (tutor_id) REFERENCES tutors(tutor_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
) ENGINE=InnoDB;

-- 5. Table lesson_reports: Post-lesson evaluation and confirmed fee (Strict 1-1 relation with lessons)
CREATE TABLE lesson_reports (
    report_id INT AUTO_INCREMENT PRIMARY KEY,
    lesson_id INT NOT NULL UNIQUE,
    content TEXT NOT NULL,
    feedback TEXT NOT NULL,
    duration_minutes INT NOT NULL,
    fee DECIMAL(12,2) NOT NULL,
    CONSTRAINT chk_report_duration CHECK (duration_minutes > 0),
    CONSTRAINT chk_report_fee CHECK (fee >= 0),
    CONSTRAINT fk_reports_lesson
        FOREIGN KEY (lesson_id) REFERENCES lessons(lesson_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
) ENGINE=InnoDB;

-- 6. Table payments: Tuition payment transactions (1-N relation with lessons)
CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    lesson_id INT NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    payment_date DATE NOT NULL,
```

```
    method ENUM('Cash', 'BankTransfer', 'Card') NOT NULL,
    CONSTRAINT chk_payment_amount CHECK (amount > 0),
    CONSTRAINT fk_payments_lesson
        FOREIGN KEY (lesson_id) REFERENCES lessons(lesson_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

SECTION 3. SAMPLE DATA & INTEGRITY VERIFICATION (PART B — SEED.SQL)

3.1. Sample Dataset Design Matrix Covering All Required Edge Cases

The sample dataset in `seed.sql` exceeds all minimum row-count thresholds and is systematically designed to cover all 10 required edge-case scenarios for testing the 10 analytical queries and 3 VIEWs:

No.	Required Data Scenario / Edge Case	Actual Coverage	Specific Records in `seed.sql`
1	Minimum row count per table	6 / 6 / 11 / 23 / 16 / 17	`subjects` =6 (≥5), `tutors` =6 (≥6), `students` =11 (≥10), `lessons` =23 (≥12), `lesson_reports` =16 (≥10), `payments` =17 (≥10).
2	Temporal diversity: Jan 2025, rest of Q1/2025 (Feb, Mar), outside Q1/2025, outside 2025	All 4 time partitions covered	• Jan 2025: `lesson_id` 1, 2, 6, 10, 22 • Feb & Mar 2025: `lesson_id` 3, 4, 7, 8, 11, 12, 15, 17, 19, 20, 21, 23 • Outside Q1/2025 (May, Jun 2025): `lesson_id` 5, 9, 13, 18 • Year 2024: `lesson_id` 14 (`2024-11-15`), 16 (`2024-12-10`).
3	All 3 lesson statuses & strict 1–1 mapping between `Completed` and `lesson_reports`	16 Completed, 4 Pending, 3 Cancelled	All 16/16 `Completed` lessons (`lesson_id` 1..16) have reports in `lesson_reports` ; 0 `Pending` / `Cancelled` lessons have reports.
4	At least 1 student with **more than 3 Completed lessons** in 2025 (≥ 4 lessons)	2 qualifying students	• `student_id = 1` (Le Hoang Minh Khoi): **5 Completed lessons** in 2025 (lessons 1, 2, 3, 4, 5) • `student_id = 2` (Nguyen Phuong Anh): **4 Completed lessons** in 2025 (lessons 6, 7, 8, 9).
5	At least 1 student with lesson history but zero payments	2 students	• `student_id = 7` (Vu Ngoc Ha): Has lesson 15 (`Completed` , fee 450,000) and lesson 18 (`Pending`), but 0 records in `payments` . • `student_id = 8` (Bui Duc Thinh): Has lessons 19, 20 (`Pending`) and 22 (`Cancelled`), 0 payments.
6	At least 1 student who has never had any lessons	2 students	`student_id = 10` (Ngo Hai Dang) and `student_id = 11` (Ly Thanh Truc) — 0 rows in `lessons` .
7	At least 1 lesson paid in multiple installments (multiple `payments` rows for the same `lesson_id`)	2 multi-installment lessons	• `lesson_id = 1` : `payment_id` 1 (300,000) + `payment_id` 2 (200,000) = 500,000 (fully paid in 2 installments). • `lesson_id = 6` : `payment_id` 7 (200,000) + `payment_id` 8 (150,000) = 350,000 / 500,000 (paid in 2 installments, owes 150,000).
8	At least 1 lesson with outstanding debt (total `amount < fee`)	4 lessons with remaining debt	`lesson_id = 6` (owes 150,000), `lesson_id = 10` (owes 200,000), `lesson_id = 13` (owes 200,000), `lesson_id = 15` (owes 450,000 — unpaid).
9	At least 1 subject with no lessons in Q1/2025	2 subjects	• `subject_id = 5` (Macroeconomics 1): only has 2 lessons in 2024. • `subject_id = 6` (General Law): 0 lessons across all time.
10	At least 1 tutor with no lessons in 2025	1 tutor	• `tutor_id = 6` (M.Sc. Pham Duc Chung): taught 2 lessons in 2024, 0 lessons in 2025.

Table 3.0: Traceability matrix mapping `seed.sql` sample data to all project requirements.

3.2. Seeded Data Across the 6 Tables

subject_id	name
1	Data Analysis with Python
2	Data Structures and Algorithms with Python
3	Introduction to Databases
4	Introduction to Artificial Intelligence
5	Macroeconomics 1
6	General Law

Table 3.1: Seeded data in the subjects table (6 records).

tutor_id	full_name	email	qualification	subject_id
1	Dr. Tran Hung	hung.tran@neu.edu.vn	Ph.D. in Data Science	1
2	Dr. Vu Duc Minh	minhvd@neu.edu.vn	Ph.D. in Computer Science	2
3	Dr. Tran Duc Minh	minhdt@neu.edu.vn	Ph.D. in Information Systems	3
4	Dr. Nguyen Trong Nghia	nghiant@neu.edu.vn	Ph.D. in Artificial Intelligence	4
5	Dr. Nguyen Phuc Hai	hainp@neu.edu.vn	Ph.D. in Data Science & AI	1
6	M.Sc. Pham Duc Chung	chungpd@neu.edu.vn	M.Sc. in Economics	5

Table 3.2: Seeded data in the tutors table (6 records).

student_id	full_name	dob	phone
1	Le Hoang Minh Khoi	2006-03-15	0912345101
2	Nguyen Phuong Anh	2006-07-22	0912345102
3	Tran Quoc Tuan	2006-01-10	0912345103
4	Pham Gia Bao	2006-11-05	0912345104
5	Hoang Mai Linh	2005-08-19	0912345105
6	Dang Tuan Kiet	2005-12-25	0912345106
7	Vu Ngoc Ha	2006-04-30	0912345107
8	Bui Duc Thinh	2006-09-12	0912345108
9	Do Khanh Huyen	2005-05-08	0912345109
10	Ngo Hai Dang	2007-02-18	0912345110
11	Ly Thanh Truc	2007-06-20	0912345111

Table 3.3: Seeded data in the students table (11 records).

lesson_id	student_id	tutor_id	lesson_date	status
1	1	1	2025-01-08	Completed
2	1	2	2025-01-18	Completed
3	1	1	2025-02-10	Completed
4	1	2	2025-03-05	Completed
5	1	3	2025-05-14	Completed
6	2	1	2025-01-22	Completed
7	2	3	2025-02-15	Completed
8	2	2	2025-03-20	Completed
9	2	4	2025-06-10	Completed
10	3	1	2025-01-25	Completed
11	3	5	2025-02-22	Completed
12	4	3	2025-03-12	Completed
13	5	2	2025-05-20	Completed
14	6	6	2024-11-15	Completed
15	7	4	2025-03-28	Completed
16	9	6	2024-12-10	Completed
17	4	5	2025-03-29	Pending
18	7	3	2025-05-25	Pending
19	8	4	2025-02-18	Pending
20	8	2	2025-03-30	Pending
21	5	1	2025-02-05	Cancelled
22	8	3	2025-01-29	Cancelled
23	3	5	2025-03-15	Cancelled

Table 3.4: Seeded data in the lessons table (23 records).

report_id	lesson_id	content	feedback	duration_minutes	fee
1	1	Multidimensional NumPy arrays and Pandas DataFrame manipulation	Student mastered array slicing and vectorized data operations	90	500000.00
2	2	Big-O algorithmic complexity analysis and Linked Lists	Implemented Singly and Doubly Linked Lists fluently in Python	120	600000.00
3	3	Data cleaning techniques and handling missing values	Applied interpolation and outlier filtering effectively on real datasets	90	500000.00
4	4	Stacks, Queues, and Recursive problem-solving in Python	Solved postfix expression evaluation and backtracking problems well	120	600000.00
5	5	Entity-Relationship Diagram (ERD) design and relational mapping	Correctly identified primary keys, foreign keys, and cardinalities	90	550000.00
6	6	Exploratory Data Analysis (EDA) and descriptive statistics with Python	Proficient in aggregating data using groupby and pivot_table	90	500000.00
7	7	Database normalization (1NF, 2NF, 3NF, BCNF) and functional dependencies	Solid grasp of attribute closure and lossless-join decomposition	90	550000.00
8	8	Binary Search Trees (BST) and tree traversal algorithms (Inorder, Preorder)	Wrote clean recursive code and understood tree balancing concepts	120	600000.00
9	9	Uninformed search strategies (BFS, DFS) and state-space formulation	Accurately modeled 2D grid pathfinding problems	90	450000.00
10	10	Advanced data visualization with Matplotlib and Seaborn	Designed clear distribution plots and correlation heatmaps	90	500000.00
11	11	Feature engineering and data scaling techniques	Practiced MinMaxScaler and StandardScaler effectively on sample data	90	480000.00
12	12	Advanced SQL queries: INNER/LEFT JOIN, GROUP BY, HAVING, and Subqueries	Wrote optimized queries and prevented chasm-trap fanout errors	90	550000.00
13	13	Sorting algorithms (QuickSort, MergeSort) and Dynamic Programming	Optimized memory usage and runtime on large input arrays	120	600000.00
14	14	Measuring National Income (GDP), Consumer Price Index (CPI), and inflation	Clearly distinguished nominal vs. real GDP and growth rates	90	400000.00
15	15	Informed search algorithms (Greedy Best-First Search, A*) and heuristics	Proved admissibility and consistency of Manhattan distance heuristic	90	450000.00
16	16	Aggregate Demand - Aggregate Supply (AD-AS) model and fiscal/monetary policy	Provided sharp analysis of interest rate and public spending impacts	90	400000.00

Table 3.5: Seeded data in the lesson_reports table (16 records, strict 1-1 relation with Completed lessons).

payment_id	lesson_id	amount	payment_date	method
1	1	300000.00	2025-01-08	Cash
2	1	200000.00	2025-01-12	BankTransfer
3	2	600000.00	2025-01-19	BankTransfer
4	3	500000.00	2025-02-10	Card
5	4	600000.00	2025-03-06	BankTransfer
6	5	550000.00	2025-05-15	Cash
7	6	200000.00	2025-01-22	Cash
8	6	150000.00	2025-01-28	BankTransfer
9	7	550000.00	2025-02-16	Card
10	8	600000.00	2025-03-21	BankTransfer
11	9	450000.00	2025-06-11	Cash
12	10	300000.00	2025-01-26	BankTransfer
13	11	480000.00	2025-02-23	Cash
14	12	550000.00	2025-03-12	Card
15	13	400000.00	2025-05-21	BankTransfer
16	14	400000.00	2024-11-16	Cash
17	16	400000.00	2024-12-11	BankTransfer

Table 3.6: Seeded data in the payments table (17 records).

3.3. Execution Results of Data Integrity Verification Queries in `seed.sql`

At the end of `seed.sql`, 4 automated reconciliation `SELECT` queries verify that the seeded dataset strictly satisfies the 1–1 reporting rule (for `Completed` lessons) and the payment cap rule (`SUM(amount) <= fee`):

status	total_lessons	total_reports
Completed	16	16
Pending	4	0
Cancelled	3	0

Table 3.7: Verification 1A Result — Lesson counts by status vs. corresponding lesson report counts.

✓ Query Result: **0 rows (Empty set)** — No constraint violations detected!

Table 3.8: Verification 1B Result — Checking for any lessons violating the 1-1 report rule.

lesson_id	status	confirmed_fee	total_paid	remaining_debt	validation_status
1	Completed	500000.00	500000.00	0.00	VALID
2	Completed	600000.00	600000.00	0.00	VALID
3	Completed	500000.00	500000.00	0.00	VALID
4	Completed	600000.00	600000.00	0.00	VALID
5	Completed	550000.00	550000.00	0.00	VALID
6	Completed	500000.00	350000.00	150000.00	VALID
7	Completed	550000.00	550000.00	0.00	VALID
8	Completed	600000.00	600000.00	0.00	VALID
9	Completed	450000.00	450000.00	0.00	VALID
10	Completed	500000.00	300000.00	200000.00	VALID
11	Completed	480000.00	480000.00	0.00	VALID
12	Completed	550000.00	550000.00	0.00	VALID
13	Completed	600000.00	400000.00	200000.00	VALID
14	Completed	400000.00	400000.00	0.00	VALID
15	Completed	450000.00	0.00	450000.00	VALID
16	Completed	400000.00	400000.00	0.00	VALID

Table 3.9: Verification 2A Result — Reconciling confirmed tuition fees and total payments per lesson.

✓ Query Result: **0 rows (Empty set)** — No constraint violations detected!

Table 3.10: Verification 2B Result — Checking for payments exceeding the confirmed fee or lacking a lesson report.

3.4. Analysis of Commented-Out Constraint Violation `INSERT` Statements

4 Negative Test Cases Included as Comments at the End of seed.sql

- **Foreign Key Violation (`FOREIGN KEY`):** `INSERT INTO lessons ... VALUES (999, 1, '2025-04-01', 'Pending');` — Rejected by `fk_lessons_student` (MySQL `ERROR 1452: Cannot add or update a child row`) because `student_id = 999` does not exist.
- **Check Constraint Violation (`duration_minutes <= 0`):** `INSERT INTO lesson_reports ... VALUES (17, 'Review Data Analysis with Python', 'Good', 0, 450000.00);` — Rejected by `chk_report_duration` (MySQL `ERROR 3819: Check constraint is violated`).
- **Check Constraint Violation (`amount <= 0`):** `INSERT INTO payments ... VALUES (1, -50000.00, '2025-01-15', 'Cash');` — Rejected by `chk_payment_amount` (MySQL `ERROR 3819`).
- **Unique Constraint Violation (Duplicate `lesson_id`):** `INSERT INTO lesson_reports (lesson_id, ...) VALUES (1, ...);` — Rejected by `UNIQUE (lesson_id)` (MySQL `ERROR 1062: Duplicate entry '1'`).

SECTION 4. 10 ANALYTICAL DATA QUERIES (PART C — QUERIES.SQL)

Core Technical Principle: Anti-Fanout / Chasm Trap Prevention

In the `tutoring_mini` schema, a lesson (`lessons`) has at most **1** tuition report (`lesson_reports.fee`) but can have **N** payment installments (`payments.amount`). Joining `lessons JOIN lesson_reports JOIN payments` directly and computing `SUM(lr.fee)` or `AVG(lr.fee)` causes multi-installment lessons (such as `lesson_id = 1` and `lesson_id = 6` , each paid in 2 installments) to duplicate `lr.fee` ! To eliminate this fanout error: (1) Queries that only aggregate tuition fees (Query 4, Query 8, `v_group_summary`) join `lesson_reports` without joining `payments` ; (2) Reconciliations that combine both fees and payments (such as `v_debt`) **pre-aggregate `payments` by `lesson_id` inside a Subquery first** to restore a 1–1 cardinality before performing the outer join.

QUERY 1 List Completed Lessons in January 2025

Requirement: List all `Completed` lessons in January 2025, including student ID + name, tutor ID + name, and `lesson_date` .

Technical Analysis & Solution Design: Perform an `INNER JOIN` from `lessons` to `students` (on `student_id`) and `tutors` (on `tutor_id`). Filter by `l.status = 'Completed'` combined with the half-open date range `l.lesson_date >= '2025-01-01' AND l.lesson_date < '2025-02-01'` (which preserves B-Tree index efficiency and is safe for both `DATE` and `DATETIME` types). Lesson 22 (on `2025-01-29` for student Bui Duc Thinh) has status `Cancelled` and is therefore accurately excluded.

```
SELECT
    l.lesson_id,
    s.student_id,
    s.full_name AS student_name,
    t.tutor_id,
    t.full_name AS tutor_name,
    l.lesson_date,
    l.status
FROM lessons l
JOIN students s ON l.student_id = s.student_id
JOIN tutors t ON l.tutor_id = t.tutor_id
WHERE l.status = 'Completed'
  AND l.lesson_date >= '2025-01-01'
  AND l.lesson_date < '2025-02-01'
ORDER BY l.lesson_date ASC, l.lesson_id ASC;
```

lesson_id	student_id	student_name	tutor_id	tutor_name	lesson_date	status
1	1	Le Hoang Minh Khoi	1	Dr. Tran Hung	2025-01-08	Completed
2	1	Le Hoang Minh Khoi	2	Dr. Vu Duc Minh	2025-01-18	Completed
6	2	Nguyen Phuong Anh	1	Dr. Tran Hung	2025-01-22	Completed
10	3	Tran Quoc Tuan	1	Dr. Tran Hung	2025-01-25	Completed

Table 4.1: Live execution result of Query 1 on MySQL.

QUERY 2 Count Total Lessons per Subject in Q1/2025

Requirement: Count the total number of lessons per subject in Q1/2025 (from `2025-01-01` through `2025-03-31`) across all statuses; include subjects with a count of `0`.

Technical Analysis & Solution Design: Start from the parent table `subjects` and perform a `LEFT JOIN` to `tutors` and then to `lessons`. **Key Technical Point:** The Q1/2025 date filter (`l.lesson_date >= '2025-01-01' AND l.lesson_date < '2025-04-01'`) must be placed directly inside the `ON` clause of the `LEFT JOIN lessons` rather than in the `WHERE` clause. Placing it in `WHERE` would convert the outer join into an inner join and eliminate subjects with zero Q1/2025 lessons (such as *Macroeconomics 1*, which only had lessons in 2024, and *General Law*, which has no lessons at all). Using `COUNT(l.lesson_id)` counts only non-NULL lesson IDs, accurately returning `0` for those two subjects.

```
SELECT
    sub.subject_id,
    sub.name AS subject_name,
    COUNT(l.lesson_id) AS total_lessons_q1_2025
FROM subjects sub
LEFT JOIN tutors t ON sub.subject_id = t.subject_id
LEFT JOIN lessons l ON t.tutor_id = l.tutor_id
    AND l.lesson_date >= '2025-01-01'
    AND l.lesson_date < '2025-04-01'
GROUP BY sub.subject_id, sub.name
ORDER BY total_lessons_q1_2025 DESC, sub.subject_id ASC;
```

subject_id	subject_name	total_lessons_q1_2025
1	Data Analysis with Python	8
2	Data Structures and Algorithms with Python	4
3	Introduction to Databases	3
4	Introduction to Artificial Intelligence	2
5	Macroeconomics 1	0
6	General Law	0

Table 4.2: Live execution result of Query 2 on MySQL.

QUERY 3 List Students with Unreported Lessons (Excluding Cancelled)

Requirement: List students who have at least one lesson without a lesson report, excluding `Cancelled` lessons. Each student must appear only once.

Technical Analysis & Solution Design: Perform an `INNER JOIN` from `students` to `lessons` and a `LEFT JOIN` to `lesson_reports`. Lessons lacking a report are identified via the anti-join condition `lr.report_id IS NULL`, while cancelled lessons are excluded with `l.status <> 'Cancelled'`. Because a student may have multiple unreported `Pending` lessons (for example, `student_id = 8` Bui Duc Thinh has 2 `Pending` lessons: 19 and 20), `SELECT DISTINCT` guarantees that each student appears at most once. Student 5 (Hoang Mai Linh) only has an unreported lesson with status `Cancelled` and is properly excluded.

```
SELECT DISTINCT
    s.student_id,
    s.full_name AS student_name
FROM students s
JOIN lessons l ON s.student_id = l.student_id
LEFT JOIN lesson_reports lr ON l.lesson_id = lr.lesson_id
WHERE l.status <> 'Cancelled'
  AND lr.report_id IS NULL
ORDER BY s.student_id ASC;
```

student_id	student_name
4	Pham Gia Bao
7	Vu Ngoc Ha
8	Bui Duc Thinh

Table 4.3: Live execution result of Query 3 on MySQL.

QUERY 4 **Average Tuition Fee per Completed Lesson by Subject**

Requirement: Calculate the average tuition fee per `Completed` lesson for each subject across all time; include only subjects that have incurred fees.

Technical Analysis & Solution Design: Use `INNER JOIN` across the chain `subjects → tutors → lessons → lesson_reports` with the filter `l.status = 'Completed'`. Because the specification requires *only subjects that have incurred fees*, `INNER JOIN` naturally excludes *General Law* (which has no completed lessons). **Anti-Fanout / Chasm Trap Prevention:** We strictly avoid joining the `payments` table here; if a lesson was paid in two installments (such as `lesson_id = 1` and `lesson_id = 6` in *Data Analysis with Python*), joining `payments` would duplicate those lessons' fee rows and distort `AVG(lr.fee)`.

```
SELECT
    sub.subject_id,
    sub.name AS subject_name,
    COUNT(lr.report_id) AS completed_lessons_count,
    ROUND(AVG(lr.fee), 2) AS avg_completed_fee
FROM subjects sub
JOIN tutors t ON sub.subject_id = t.subject_id
JOIN lessons l ON t.tutor_id = l.tutor_id
JOIN lesson_reports lr ON l.lesson_id = lr.lesson_id
WHERE l.status = 'Completed'
GROUP BY sub.subject_id, sub.name
ORDER BY avg_completed_fee DESC, sub.subject_id ASC;
```

subject_id	subject_name	completed_lessons_count	avg_completed_fee
2	Data Structures and Algorithms with Python	4	600000.00
3	Introduction to Databases	3	550000.00
1	Data Analysis with Python	5	496000.00
4	Introduction to Artificial Intelligence	2	450000.00
5	Macroeconomics 1	2	400000.00

Table 4.4: Live execution result of Query 4 on MySQL.

QUERY 5 **Top 3 Tutors with the Most Completed Lessons in 2025**

Requirement: List the top 3 tutors with the highest number of `Completed` lessons in 2025. Break ties by `tutor_id` ascending; include only tutors with activity.

Technical Analysis & Solution Design: Use an `INNER JOIN` between `tutors` and `lessons` filtered by `l.status = 'Completed'` and the 2025 date range (`l.lesson_date >= '2025-01-01' AND l.lesson_date < '2026-01-01'`) so that only tutors with completed lessons in 2025 are considered. Group by `t.tutor_id, t.full_name` and sort using `ORDER BY completed_lessons_2025 DESC, t.tutor_id ASC LIMIT 3`. In the dataset, Tutor 1 (Dr. Tran Hung) and Tutor 2 (Dr. Vu Duc Minh) both have 4 `Completed` lessons in 2025, and the tie-breaker orders `tutor_id = 1` first.

```
SELECT
    t.tutor_id,
    t.full_name AS tutor_name,
    COUNT(l.lesson_id) AS completed_lessons_2025
FROM tutors t
JOIN lessons l ON t.tutor_id = l.tutor_id
WHERE l.status = 'Completed'
  AND l.lesson_date >= '2025-01-01'
  AND l.lesson_date < '2026-01-01'
GROUP BY t.tutor_id, t.full_name
ORDER BY completed_lessons_2025 DESC, t.tutor_id ASC
LIMIT 3;
```

tutor_id	tutor_name	completed_lessons_2025
1	Dr. Tran Hung	4
2	Dr. Vu Duc Minh	4
3	Dr. Tran Duc Minh	3

Table 4.5: Live execution result of Query 5 on MySQL.

QUERY 6 Total Amount Paid by Each Student in 2025

Requirement: Calculate the total amount paid by each student in 2025 based on `payment_date`; display `0` for students who made no payments.

Technical Analysis & Solution Design: Start from `students` and perform a `LEFT JOIN` to `lessons` and `LEFT JOIN` to `payments`. The 2025 payment date filter (`p.payment_date >= '2025-01-01' AND p.payment_date < '2026-01-01'`) is placed inside the `ON` clause of `LEFT JOIN payments`. Consequently, students who only paid in 2024 (Student 6 Dang Tuan Kiet, Student 9 Do Khanh Huyen), students with 100% unpaid balance (Student 7 Vu Ngoc Ha), and students who never took a lesson (Student 10 Ngo Hai Dang, Student 11 Ly Thanh Truc) are all preserved and displayed with `0.00` via `COALESCE(SUM(p.amount), 0.00)`.

```
SELECT
    s.student_id,
    s.full_name AS student_name,
    COALESCE(SUM(p.amount), 0.00) AS total_paid_2025
FROM students s
LEFT JOIN lessons l ON s.student_id = l.student_id
LEFT JOIN payments p ON l.lesson_id = p.lesson_id
    AND p.payment_date >= '2025-01-01'
    AND p.payment_date < '2026-01-01'
GROUP BY s.student_id, s.full_name
ORDER BY total_paid_2025 DESC, s.student_id ASC;
```

student_id	student_name	total_paid_2025
1	Le Hoang Minh Khoi	2750000.00
2	Nguyen Phuong Anh	1950000.00
3	Tran Quoc Tuan	780000.00
4	Pham Gia Bao	550000.00
5	Hoang Mai Linh	400000.00
6	Dang Tuan Kiet	0.00
7	Vu Ngoc Ha	0.00
8	Bui Duc Thinh	0.00
9	Do Khanh Huyen	0.00
10	Ngo Hai Dang	0.00
11	Ly Thanh Truc	0.00

Table 4.6: Live execution result of Query 6 on MySQL.

QUERY 7 **List Students with Zero Payment Records Across All Time**

Requirement: List all students who do not have any payment records across the entire dataset, including students who have never had a lesson.

Technical Analysis & Solution Design: Perform a `LEFT JOIN` from `students` to `lessons` and `payments` across all time. Group by student and filter using `HAVING COUNT(p.payment_id) = 0`. The result captures both edge-case groups: (1) Students who have lesson history but have never made a payment (Vu Ngoc Ha `student_id = 7`, Bui Duc Thinh `student_id = 8`) and (2) Newly registered students who have never booked a lesson (Ngo Hai Dang `student_id = 10`, Ly Thanh Truc `student_id = 11`).

```
SELECT
    s.student_id,
    s.full_name AS student_name,
    s.dob,
    s.phone
FROM students s
LEFT JOIN lessons l ON s.student_id = l.student_id
LEFT JOIN payments p ON l.lesson_id = p.lesson_id
GROUP BY s.student_id, s.full_name, s.dob, s.phone
HAVING COUNT(p.payment_id) = 0
ORDER BY s.student_id ASC;
```

student_id	student_name	dob	phone
7	Vu Ngoc Ha	2006-04-30	0912345107
8	Bui Duc Thinh	2006-09-12	0912345108
10	Ngo Hai Dang	2007-02-18	0912345110
11	Ly Thanh Truc	2007-06-20	0912345111

Table 4.7: Live execution result of Query 7 on MySQL.

QUERY 8 **Rank All Students by Total Confirmed Tuition Fee Descending**

Requirement: List all students ordered by total confirmed tuition fee descending, regardless of whether they have paid; break ties by `student_id` ascending.

Technical Analysis & Solution Design: Anti-Fanout / Chasm Trap Prevention: Confirmed tuition fees reside in `lesson_reports` (which has a strict 1–1 relationship with `lessons` via `lesson_id UNIQUE`). Thus, we perform a `LEFT JOIN` from `students` to `lessons` (with `l.status = 'Completed'`) and `lesson_reports` without joining `payments`. If `payments` were joined directly, Le Hoang Minh Khoi (`student_id = 1`, who paid for Lesson 1 in 2 installments) would have Lesson 1's fee of 500,000 VND double-counted. Students tied in total confirmed fee (e.g., Students 6 and 9 at `400000.00`; Students 8, 10, 11 at `0.00`) are ordered deterministically by `student_id ASC`.

```
SELECT
    s.student_id,
    s.full_name AS student_name,
    COALESCE(SUM(lr.fee), 0.00) AS total_confirmed_fee
FROM students s
LEFT JOIN lessons l ON s.student_id = l.student_id
    AND l.status = 'Completed'
LEFT JOIN lesson_reports lr ON l.lesson_id = lr.lesson_id
GROUP BY s.student_id, s.full_name
ORDER BY total_confirmed_fee DESC, s.student_id ASC;
```

student_id	student_name	total_confirmed_fee
1	Le Hoang Minh Khoi	2750000.00
2	Nguyen Phuong Anh	2100000.00
3	Tran Quoc Tuan	980000.00
5	Hoang Mai Linh	600000.00
4	Pham Gia Bao	550000.00
7	Vu Ngoc Ha	450000.00
6	Dang Tuan Kiet	400000.00
9	Do Khanh Huyen	400000.00
8	Bui Duc Thinh	0.00
10	Ngo Hai Dang	0.00
11	Ly Thanh Truc	0.00

Table 4.8: Live execution result of Query 8 on MySQL.

QUERY 9 **Calculate Each Tutor's Lesson Cancellation Rate in 2025**

Requirement: Calculate the cancellation rate for each tutor in 2025: `(Cancelled lessons / total lessons in 2025) * 100`. Round to 2 decimal places; return `NULL` if total lessons is `0`.

Technical Analysis & Solution Design: Use a `LEFT JOIN` from `tutors` to `lessons` with the 2025 date range inside the `ON` clause so tutors with no lessons in 2025 are retained (Tutor 6 M.Sc. Pham Duc Chung only taught in 2024). Count cancelled lessons conditionally via `SUM(CASE WHEN l.status = 'Cancelled' THEN 1 ELSE 0 END)`. To prevent division by zero and return `NULL` when a tutor has 0 lessons in 2025, use `NULLIF(COUNT(l.lesson_id), 0)` in the denominator wrapped inside `ROUND(..., 2)`.

```
SELECT
    t.tutor_id,
    t.full_name AS tutor_name,
    COUNT(l.lesson_id) AS total_lessons_2025,
    SUM(CASE WHEN l.status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_lessons_2025,
    ROUND(
        SUM(CASE WHEN l.status = 'Cancelled' THEN 1 ELSE 0 END) * 100.0
        / NULLIF(COUNT(l.lesson_id), 0),
        2
    ) AS cancel_rate_2025_pct
FROM tutors t
LEFT JOIN lessons l ON t.tutor_id = l.tutor_id
    AND l.lesson_date >= '2025-01-01'
    AND l.lesson_date < '2026-01-01'
GROUP BY t.tutor_id, t.full_name
ORDER BY t.tutor_id ASC;
```

tutor_id	tutor_name	total_lessons_2025	cancelled_lessons_2025	cancel_rate_2025_pct
1	Dr. Tran Hung	5	1	20.00
2	Dr. Vu Duc Minh	5	0	0.00
3	Dr. Tran Duc Minh	5	1	20.00
4	Dr. Nguyen Trong Nghia	3	0	0.00
5	Dr. Nguyen Phuc Hai	3	1	33.33
6	M.Sc. Pham Duc Chung	0	0	*NULL*

Table 4.9: Live execution result of Query 9 on MySQL.

QUERY 10 **List Students with More Than 3 Completed Lessons in 2025**

Requirement: List students who have more than 3 `Completed` lessons in 2025, along with their lesson count; order by count descending.

Technical Analysis & Solution Design: Perform an `INNER JOIN` between `students` and `lessons`, filtering for `l.status = 'Completed'` and the 2025 date range in the `WHERE` clause. Group by student (`GROUP BY s.student_id, s.full_name`) and apply the post-aggregation filter `HAVING COUNT(l.lesson_id) > 3` (i.e., 4 or more completed lessons). The query accurately returns the 2 qualifying students: Le Hoang Minh Khoi (5 lessons) and Nguyen Phuong Anh (4 lessons).

```
SELECT
    s.student_id,
    s.full_name AS student_name,
    COUNT(l.lesson_id) AS completed_lessons_2025
FROM students s
JOIN lessons l ON s.student_id = l.student_id
WHERE l.status = 'Completed'
  AND l.lesson_date >= '2025-01-01'
  AND l.lesson_date < '2026-01-01'
GROUP BY s.student_id, s.full_name
HAVING COUNT(l.lesson_id) > 3
ORDER BY completed_lessons_2025 DESC, s.student_id ASC;
```

student_id	student_name	completed_lessons_2025
1	Le Hoang Minh Khoi	5
2	Nguyen Phuong Anh	4

Table 4.10: Live execution result of Query 10 on MySQL.

SECTION 5. ALL-TIME ANALYTICAL VIEWS (PART D — VIEWS.SQL)

The three views (`VIEW`) in `views.sql` operate across the entire historical timeline (covering both 2024 and 2025 records), providing standardized reporting interfaces for executive management and student accounts receivable:

5.1. Design and Live Output of `v_detail`

Design Logic: View `v_detail` performs an `INNER JOIN` from `lessons` to `students`, `tutors`, and `subjects` to retrieve both IDs and names for students, tutors, and subjects, followed by a `LEFT JOIN` to `lesson_reports` so that `Pending` and `Cancelled` lessons without a report are preserved (displaying `fee` as `NULL`).

```
CREATE OR REPLACE VIEW v_detail AS
SELECT
    l.lesson_id,
    l.lesson_date,
    l.status,
    s.student_id,
    s.full_name AS student_name,
    t.tutor_id,
    t.full_name AS tutor_name,
    sub.subject_id,
    sub.name AS subject_name,
    lr.fee
FROM lessons l
JOIN students s ON l.student_id = s.student_id
JOIN tutors t ON l.tutor_id = t.tutor_id
JOIN subjects sub ON t.subject_id = sub.subject_id
LEFT JOIN lesson_reports lr ON l.lesson_id = lr.lesson_id;
```

lesson_id	lesson_date	status	student_id	student_name	tutor_id	tutor_name	subject_id	subject_name	fee
1	2025-01-08	Completed	1	Le Hoang Minh Khoi	1	Dr. Tran Hung	1	Data Analysis with Python	500000.00
2	2025-01-18	Completed	1	Le Hoang Minh Khoi	2	Dr. Vu Duc Minh	2	Data Structures and Algorithms with Python	600000.00
3	2025-02-10	Completed	1	Le Hoang Minh Khoi	1	Dr. Tran Hung	1	Data Analysis with Python	500000.00
4	2025-03-05	Completed	1	Le Hoang Minh Khoi	2	Dr. Vu Duc Minh	2	Data Structures and Algorithms with Python	600000.00
5	2025-05-14	Completed	1	Le Hoang Minh Khoi	3	Dr. Tran Duc Minh	3	Introduction to Databases	550000.00
6	2025-01-22	Completed	2	Nguyen Phuong Anh	1	Dr. Tran Hung	1	Data Analysis with Python	500000.00
7	2025-02-15	Completed	2	Nguyen Phuong Anh	3	Dr. Tran Duc Minh	3	Introduction to Databases	550000.00
8	2025-03-20	Completed	2	Nguyen Phuong Anh	2	Dr. Vu Duc Minh	2	Data Structures and Algorithms with Python	600000.00
9	2025-06-10	Completed	2	Nguyen Phuong Anh	4	Dr. Nguyen Trong Nghia	4	Introduction to Artificial Intelligence	450000.00
10	2025-01-25	Completed	3	Tran Quoc Tuan	1	Dr. Tran Hung	1	Data Analysis with Python	500000.00
11	2025-02-22	Completed	3	Tran Quoc Tuan	5	Dr. Nguyen Phuc Hai	1	Data Analysis with Python	480000.00
12	2025-03-12	Completed	4	Pham Gia Bao	3	Dr. Tran Duc Minh	3	Introduction to Databases	550000.00
13	2025-05-20	Completed	5	Hoang Mai Linh	2	Dr. Vu Duc Minh	2	Data Structures and Algorithms with Python	600000.00
14	2024-11-15	Completed	6	Dang Tuan Kiet	6	M.Sc. Pham Duc Chung	5	Macroeconomics 1	400000.00
15	2025-03-28	Completed	7	Vu Ngoc Ha	4	Dr. Nguyen Trong Nghia	4	Introduction to Artificial Intelligence	450000.00
16	2024-12-10	Completed	9	Do Khanh Huyen	6	M.Sc. Pham Duc Chung	5	Macroeconomics 1	400000.00
17	2025-03-29	Pending	4	Pham Gia Bao	5	Dr. Nguyen Phuc Hai	1	Data Analysis with Python	*NULL*
18	2025-05-25	Pending	7	Vu Ngoc Ha	3	Dr. Tran Duc Minh	3	Introduction to Databases	*NULL*
19	2025-02-18	Pending	8	Bui Duc Thinh	4	Dr. Nguyen Trong Nghia	4	Introduction to Artificial Intelligence	*NULL*
20	2025-03-30	Pending	8	Bui Duc Thinh	2	Dr. Vu Duc Minh	2	Data Structures and Algorithms with Python	*NULL*
21	2025-02-05	Cancelled	5	Hoang Mai Linh	1	Dr. Tran Hung	1	Data Analysis with Python	*NULL*
22	2025-01-29	Cancelled	8	Bui Duc Thinh	3	Dr. Tran Duc Minh	3	Introduction to Databases	*NULL*
23	2025-03-15	Cancelled	3	Tran Quoc Tuan	5	Dr. Nguyen Phuc Hai	1	Data Analysis with Python	*NULL*

Table 5.1: Live query result from VIEW v_detail (all 23 lessons across all time, retaining fee = NULL for unreported lessons).

5.2. Design and Live Output of `v_group_summary`

Design Logic: View `v_group_summary` starts from `subjects` and chains `LEFT JOIN` operations through `tutors → lessons → lesson_reports`. Consequently, subjects that have never had any lessons (*General Law*, `subject_id = 6`) are retained with zero counts and `total_fee = 0.00`. Omitting the `payments` table guarantees that `SUM(lr.fee)` is free of installment fanout.

```
CREATE OR REPLACE VIEW v_group_summary AS
SELECT
    sub.subject_id,
    sub.name AS subject_name,
    COUNT(l.lesson_id) AS total_lessons,
    COALESCE(SUM(CASE WHEN l.status = 'Completed' THEN 1 ELSE 0 END), 0) AS completed_lessons,
    COALESCE(SUM(CASE WHEN l.status = 'Cancelled' THEN 1 ELSE 0 END), 0) AS cancelled_lessons,
    COALESCE(SUM(lr.fee), 0.00) AS total_fee
FROM subjects sub
LEFT JOIN tutors t ON sub.subject_id = t.subject_id
LEFT JOIN lessons l ON t.tutor_id = l.tutor_id
LEFT JOIN lesson_reports lr ON l.lesson_id = lr.lesson_id
GROUP BY sub.subject_id, sub.name;
```

subject_id	subject_name	total_lessons	completed_lessons	cancelled_lessons	total_fee
1	Data Analysis with Python	8	5	2	2480000.00
2	Data Structures and Algorithms with Python	5	4	0	2400000.00
3	Introduction to Databases	5	3	1	1650000.00
4	Introduction to Artificial Intelligence	3	2	0	900000.00
5	Macroeconomics 1	2	2	0	800000.00
6	General Law	0	0	0	0.00

Table 5.2: Live query result from VIEW v_group_summary (all-time summary across all 6 subjects).

5.3. Design and Live Output of `v_debt` (Anti-Fanout Protection)

Design Logic: View `v_debt` summarizes tuition obligations and payments for all 11 students. To **prevent tuition fee duplication** when a lesson is paid across multiple installments (such as `lesson_id = 1` for Le Hoang Minh Khoi and `lesson_id = 6` for Nguyen Phuong Anh), the derived table `pay_per_lesson` pre-aggregates `SUM(amount)` grouped by `lesson_id` before left-joining with `lessons` and `lesson_reports`:

```
CREATE OR REPLACE VIEW v_debt AS
SELECT
    s.student_id,
    s.full_name AS student_name,
    COALESCE(SUM(lr.fee), 0.00) AS total_confirmed_fee,
    COALESCE(SUM(pay_per_lesson.paid_amount), 0.00) AS total_paid,
    COALESCE(SUM(lr.fee), 0.00) - COALESCE(SUM(pay_per_lesson.paid_amount), 0.00) AS remaining_debt
FROM students s
LEFT JOIN lessons l
    ON s.student_id = l.student_id
LEFT JOIN lesson_reports lr
    ON l.lesson_id = lr.lesson_id
LEFT JOIN (
    SELECT
        lesson_id,
        SUM(amount) AS paid_amount
    FROM payments
    GROUP BY lesson_id
) pay_per_lesson
    ON l.lesson_id = pay_per_lesson.lesson_id
GROUP BY s.student_id, s.full_name;
```

student_id	student_name	total_confirmed_fee	total_paid	remaining_debt
7	Vu Ngoc Ha	450000.00	0.00	450000.00
3	Tran Quoc Tuan	980000.00	780000.00	200000.00
5	Hoang Mai Linh	600000.00	400000.00	200000.00
2	Nguyen Phuong Anh	2100000.00	1950000.00	150000.00
1	Le Hoang Minh Khoi	2750000.00	2750000.00	0.00
4	Pham Gia Bao	550000.00	550000.00	0.00
6	Dang Tuan Kiet	400000.00	400000.00	0.00
8	Bui Duc Thinh	0.00	0.00	0.00
9	Do Khanh Huyen	400000.00	400000.00	0.00
10	Ngo Hai Dang	0.00	0.00	0.00
11	Ly Thanh Truc	0.00	0.00	0.00

Table 5.3: Live query result from VIEW v_debt (accounts receivable summary for all 11 students with anti-fanout protection).